

TASK 5

Python Lists, Tuples, Sets and Dictionaries

Complete Assessment

A clear, expanded and submission-ready practical assessment covering Python collection data structures, their operations, real-world data processing, comparison, and interview preparation.

Objective

Develop practical skills for handling structured data using Python collections.

Tools

Python • Jupyter Notebook

Deliverables

Examples of all four collection types, programs demonstrating common methods, practical data-processing examples, a comparison table, and interview questions with answers.

1. TASK DESCRIPTION

Work with Python collection data structures and perform operations such as adding, removing, searching and updating elements. Lists, tuples, sets and dictionaries are fundamental Python structures used to organize and process data efficiently.

2. PYTHON COLLECTIONS OVERVIEW

Python provides several built-in collection types. Each type is designed for a different way of storing and accessing information.

- **List:** Ordered and mutable collection. It is useful when items may need to be added, removed or changed.
- **Tuple:** Ordered and immutable collection. It is useful for fixed groups of values.
- **Set:** Collection of unique elements. It is useful for removing duplicates and performing set operations.
- **Dictionary:** Key-value mapping. It is useful for structured records where values are accessed using keys.

3. LISTS — BASIC OPERATIONS

A list can contain numbers, strings, or other Python objects. Lists are indexed starting from 0 and can be modified after creation.

```
# Creating a list
students = ["Rahul", "Priya", "Aman", "Neha"]
print("Original list:", students)

# Add an element
students.append("Rohit")
print("After append:", students)

# Remove an element
students.remove("Aman")
print("After remove:", students)

# Sort the list
students.sort()
print("After sort:", students)

# Access elements
print("First student:", students[0])
print("Last student:", students[-1])
print("Number of students:", len(students))
```

Expected concept: **append()** adds an item, **remove()** removes a matching item, **sort()** orders the list, and indexing accesses individual elements.

4. LISTS — SEARCHING, UPDATING AND SLICING

```
students = ["Rahul", "Priya", "Aman", "Neha", "Rohit"]

# Searching
if "Priya" in students:
    print("Priya is present")

# Updating
students[0] = "Rahul Kumar"
print("Updated list:", students)

# Slicing
print("First three:", students[:3])
print("Last two:", students[-2:])

# Loop through the list
for name in students:
    print("Student:", name)
```

5. LIST WITH NUMERICAL DATA

```
marks = [85, 92, 67, 76, 55]

print("Total:", sum(marks))
print("Highest:", max(marks))
print("Lowest:", min(marks))
print("Average:", round(sum(marks) / len(marks), 2))

# Add a new mark
marks.append(88)

# Sort marks
marks.sort()
print("Sorted marks:", marks)
```

6. TUPLES — BASIC OPERATIONS

A tuple is ordered like a list but cannot be changed after creation. This makes tuples appropriate for fixed information.

```
student_marks = (85, 92, 67, 76, 55)

print("Tuple:", student_marks)
print("First mark:", student_marks[0])
print("Highest:", max(student_marks))
print("Lowest:", min(student_marks))
print("Total:", sum(student_marks))
print("Count of 85:", student_marks.count(85))
print("Position of 92:", student_marks.index(92))
```

Important: attempting to assign a new value to an existing tuple position raises a `TypeError` because tuples are immutable.

7. SETS — UNIQUE VALUES

A set automatically stores only unique values. It is especially useful when duplicate records or repeated values must be removed.

```
marks = {85, 92, 67, 76, 55, 85, 92}

print("Unique marks:", marks)
print("Number of unique marks:", len(marks))

# Add and remove values
marks.add(88)
marks.discard(67)

print("Updated set:", marks)
```

8. SET OPERATIONS

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

# Union: all elements from both sets
print("Union:", A | B)

# Intersection: common elements
print("Intersection:", A & B)

# Difference: elements only in A
print("A - B:", A - B)

# Symmetric difference: elements in either set but not both
print("Symmetric difference:", A ^ B)
```

9. REMOVING DUPLICATES FROM A LIST

```
marks = [85, 92, 67, 76, 55, 85, 92, 67]

unique_marks = list(set(marks))

print("Original:", marks)
print("Unique:", unique_marks)
```

10. DICTIONARIES — KEY-VALUE DATA

A dictionary stores information using keys and values. It is particularly useful for representing records such as students, employees, products and customers.

```
student = {
    "name": "Abhishek",
    "course": "B.Tech CSE",
    "semester": 8,
    "marks": 82
}

print("Name:", student["name"])
print("Course:", student["course"])
print("Marks:", student["marks"])

# Update a value
student["marks"] = 88

# Add new values
student["grade"] = "A"
student["city"] = "Fatehpur"

print("Updated record:", student)
```

11. DICTIONARY METHODS

```
student = {
    "name": "Abhishek",
    "course": "B.Tech CSE",
    "marks": 88,
    "grade": "A"
}

print("Keys:", list(student.keys()))
print("Values:", list(student.values()))
print("Items:", list(student.items()))

# Safe access
print("Name:", student.get("name"))
print("Phone:", student.get("phone", "Not available"))

# Update several values
student.update({"semester": 8, "skill": "Python"})

# Remove a key
student.pop("grade")

print("Final record:", student)
```

12. NESTED DICTIONARIES

```
employees = {
    "E101": {
        "name": "Amit",
        "department": "IT",
        "salary": 65000
    },
    "E102": {
        "name": "Sneha",
        "department": "HR",
        "salary": 58000
    },
    "E103": {
        "name": "Karan",
        "department": "IT",
        "salary": 82000
    },
    "E104": {
        "name": "Pooja",
        "department": "Finance",
        "salary": 72000
    }
}

print("Karan salary:", employees["E103"]["salary"])
```

13. EMPLOYEE SALARY FILTERING

```
for emp_id, data in employees.items():
    if data["salary"] >= 70000:
        print(emp_id, data["name"], data["salary"])
```

14. PRACTICAL STUDENT DATA PROCESSING

A list of dictionaries can represent a small structured dataset. Each dictionary represents one student record.

```
student_records = [
    {"name": "Rahul", "marks": 85},
    {"name": "Priya", "marks": 92},
    {"name": "Aman", "marks": 67},
    {"name": "Neha", "marks": 76},
    {"name": "Rohit", "marks": 55}
]

for record in student_records:
    if record["marks"] >= 80:
        record["grade"] = "A"
    elif record["marks"] >= 60:
        record["grade"] = "B"
    else:
        record["grade"] = "C"

for record in student_records:
    print(record)
```

15. FINDING TOP STUDENTS

```
top_students = []

for record in student_records:
    if record["marks"] >= 80:
        top_students.append(record["name"])

print("Top students:", top_students)
```

16. CALCULATING STUDENT AVERAGE

```
total = 0

for record in student_records:
    total += record["marks"]

average = total / len(student_records)

print("Total marks:", total)
print("Average marks:", round(average, 2))
```

17. COMPLETE COLLECTION PRACTICE

```
# List
subjects = ["Python", "SQL", "Excel"]
subjects.append("Power BI")

# Tuple
coordinates = (28.61, 77.21)

# Set
skills = {"Python", "SQL", "Python", "Pandas"}

# Dictionary
profile = {
    "name": "Abhishek",
    "skills": skills,
    "subjects": subjects
}

print(subjects)
print(coordinates)
print(skills)
print(profile)
```

18. COMPARISON TABLE

Collection	Ordered	Mutable	Duplicates	Access / Purpose
List	Yes	Yes	Allowed	Index; dynamic sequence
Tuple	Yes	No	Allowed	Index; fixed sequence
Set	No*	Yes	Not allowed	Membership / unique values
Dictionary	Insertion order	Yes	Keys unique	Key; structured records

Sets are unordered collections. Modern Python dictionaries preserve insertion order.

19. WHEN TO USE EACH COLLECTION

- **Use a list** when order matters and the collection will change.
- **Use a tuple** when the values should remain fixed.
- **Use a set** when uniqueness and fast membership checks are important.
- **Use a dictionary** when each value should be associated with a meaningful key.

20. INTERVIEW QUESTIONS AND ANSWERS

Q1. What is the difference between list, tuple, set and dictionary?

A list is ordered and mutable; a tuple is ordered and immutable; a set stores unique elements; and a dictionary stores key-value pairs.

Q2. Why are sets useful for unique values?

Sets automatically eliminate duplicate elements and support operations such as union and intersection.

Q3. How do you access a dictionary value?

Use `dictionary[key]` or `dictionary.get(key)`. The `get()` method can provide a default value when the key is missing.

Q4. Can a tuple be modified?

No. Tuples are immutable after creation.

Q5. Which collection is best for key-value data?

A dictionary.

Q6. What does `append()` do?

It adds an element to the end of a list.

Q7. What is the difference between `remove()` and `pop()`?

For lists, `remove()` deletes the first matching value, while `pop()` removes an item by index and returns it. Dictionary `pop()` removes a key and returns its value.

21. COMPLETE INTEGRATED PROGRAM

The following program combines the four collection types and applies them to simple data-processing tasks.

```
# LIST
students = ["Rahul", "Priya", "Aman", "Neha"]
students.append("Rohit")
students.remove("Aman")
students.sort()
print("Students:", students)

# TUPLE
marks = (85, 92, 67, 76, 55)
print("Highest mark:", max(marks))
print("Average:", round(sum(marks) / len(marks), 2))

# SET
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print("Union:", A | B)
print("Intersection:", A & B)

# DICTIONARY
student = {
    "name": "Abhishek",
    "course": "B.Tech CSE",
    "marks": 82
}
student["marks"] = 88
student["grade"] = "A"
print("Student:", student)

# LIST OF DICTIONARIES
records = [
    {"name": "Rahul", "marks": 85},
    {"name": "Priya", "marks": 92},
    {"name": "Aman", "marks": 67},
    {"name": "Neha", "marks": 76},
    {"name": "Rohit", "marks": 55}
]

# Grade classification
for record in records:
    if record["marks"] >= 80:
        record["grade"] = "A"
    elif record["marks"] >= 60:
        record["grade"] = "B"
    else:
        record["grade"] = "C"

print("Student records:")
for record in records:
    print(record)
```